

Тема 1.3. Міри складності алгоритмів

Для розв'язування однієї й тієї ж задачі можна застосувати різні алгоритми. Ефективність роботи кожного з них визначається різними факторами і залежить від:

- кваліфікації програміста,
- потужності обчислювальних засобів,
- застосовуваних методів обчислень.

При аналізі алгоритму визначається кількість «часу», необхідного для його виконання.

Критерій часової оцінки – це не реальне число секунд або інших проміжків часу, а приблизне число операцій, виконуваних алгоритмом. Число операцій визначає відносний час виконання алгоритму. Таким чином, іноді ми будемо називати «часом» середню кількість операцій, необхідних для виконання алгоритму. Фактична кількість секунд, необхідних для виконання алгоритму на комп'ютері, непридатна для аналізу, оскільки нас цікавить тільки відносна ефективність алгоритму, що вирішує конкретну задачу. Дійсно, алгоритм не покращується, якщо його перенести на більш швидкий комп'ютер, і не погіршується, якщо його виконувати на більш повільному комп'ютері.

Крім того, фактична кількість операцій алгоритму на тих або інших даних, що вводяться, не надає вичерпної інформації про якість алгоритму або мало його характеризує. Замість цього важливою характеристикою є залежність числа операцій конкретного алгоритму від обсягу вхідних даних. Ми можемо порівняти два алгоритми за швидкістю зростання числа операцій у залежності від зростання обсягу вхідних даних. Саме швидкість зростання відіграє ключову роль.

При аналізі алгоритмів враховують складність алгоритмів за часом, однак потрібно враховувати й те, який обсяг пам'яті потрібний для роботи того або іншого алгоритму. На ранніх етапах розвитку комп'ютерів цей аналіз

носив принциповий характер. Досить часто доводилося вибирати більш повільний алгоритм, якщо він вимагав менше пам'яті. Розроблювачі сучасних програм не відчують потреби в економії пам'яті, у результаті чого комп'ютер морально застаріває задовго до його фізичної непридатності.

Швидкість росту алгоритму визначається швидкістю зростання числа операцій при зростанні обсягу вхідних даних. Нас цікавить тільки загальний характер поведінки алгоритму, а не подробиці цієї поведінки. Отже, при аналізі алгоритмів нас буде цікавити швидше за все клас функції швидкості росту, до якого належить алгоритм, ніж точна кількість виконуваних ним операцій [7].

Деякі поширені класи функцій наведені в таблиці. У цій таблиці наведені значення функцій з даного класу на широкому діапазоні значень аргументу. Видно, що при невеликих обсягах вхідних даних значення функцій мало відрізняються, однак при зростанні цих обсягів різниця суттєво зростає. Крім того, швидкодіючі функції домінують над функціями з більш повільним ростом. Тому якщо ми виявимо, що складність алгоритму є сумою двох або декількох таких функцій, то будемо часто відкидати всі функції, крім тих, які зростають найшвидше. Якщо, наприклад, буде встановлено, що алгоритму потрібно $x^3 - 30x$ операцій, то будемо вважати, що складність алгоритму росте як x^3 , оскільки вже при 100 вхідних даних різниця між x^3 і $x^3 - 30x$ становить лише 0,3%.

Таблиця 1

Таблиця класів росту функцій

n	$\log_2 n$	n^2	n^3	2^n	$n!$
1	0	1	1	2	1
2	1	4	8	4	2
5	2.3	25	125	32	120
10	3.3	100	1000	1024	362880
15	3.9	225	3375	32768	-----
20	4.3	400	8000	1048576	-----
30	4.9	900	27000	1073741824	-----

Швидкість росту складності алгоритму відіграє важливу роль, швидкість росту визначається старшим членом формули. Відкинувши всі молодші члени, ми одержуємо те, що називають порядком функції або алгоритму, швидкістю росту складності якого вона є. Алгоритми можна згрупувати за швидкостями росту їх складностей. Введемо насамперед деякі поняття, пов'язані з асимптотичною оцінкою функцій.

Асимптотична обчислювальна складність

Хоча в багатьох випадках ці оцінки використовують неформально, корисно почати з точних визначень.

Зазначимо, що у даному розділі функції відображають цілі числа в цілі, у тому випадку, якщо з контексту це незрозуміло [8].

Оцінка Θ . Якщо $f(n)$ й $g(n)$ – деякі функції, то запис $f(n) = \Theta(g(n))$ означає, що знайдуться такі $c_1, c_2 > 0$ й таке n_0 , що $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ для всіх $n \geq n_0$.

У цьому випадку говорять, що $g(n)$ є асимптотично точною оцінкою для $f(n)$.

Однак це позначення слід використовувати з обережністю. Встановивши, що $f_1(n) = \Theta(g(n))$ й $f_2(n) = \Theta(g(n))$, не слід робити висновок, що $f_1(n) = f_2(n)$!

Визначення $\Theta(g(n))$ припускає, що функції $f(n)$ й $g(n)$ асимптотично невід'ємні, тобто невід'ємні для достатньо великих значень n . Зазначимо, що якщо f й g строго додатні, то можна виключити n_0 з визначення (змінивши константи c_1 й c_2 так, щоб для малих n нерівність також виконувалася).

Це відношення симетричне: якщо $f(n) = \Theta(g(n))$, то $g(n) = \Theta(f(n))$.

Приклад 1. Доведення асимптотичної складності

Перевіримо, що $\frac{n^2}{2} - 3n = \Theta(n^2)$. Згідно з визначенням, потрібно вказати додатні константи c_1, c_2 і число n_0 так, щоб нерівності

$$c_1 n^2 \leq \frac{n^2}{2} - 3n \leq c_2 n^2$$

виконувалися для всіх $n \geq n_0$. Розділимо згадані нерівності на n^2 :

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$$

Видно, що для виконання другої нерівності $\frac{1}{2} - \frac{3}{n} \leq c_2$ достатньо прийняти, що $c_2 = \frac{1}{2}$. Перша нерівність $c_1 \leq \frac{1}{2} - \frac{3}{n}$ буде виконана, якщо, наприклад, $n_0 = 7$ і $c_1 = \frac{1}{14}$.

Приклад 2. Доведення нерівності асимптотичної складності Використання формального визначення: покажемо, що $6n^3 \neq \Theta(n^2)$. Насправді, припустимо, що найдуться такі c_2 й n_0 , що $6n^3 \leq c_2 n^2$ для всіх $n \geq n_0$. Розділивши нерівність $6n^3 \leq c_2 n^2$ на n^2 , одержуємо $6n \leq c_2$, звідси $n \leq \frac{c_2}{6}$. Яке б велике додатне c_2 ми б не взяли, завжди знайдеться таке додатне n , значення якого перевищить $\frac{c_2}{6}$.

Відшукуючи асимптотично точну оцінку для суми, ми можемо відкидати члени меншого порядку, які при більших n стають малими в порівнянні з основним доданком. Відзначимо також, що коефіцієнт при старшому члені не впливає (він може вплинути тільки на вибір констант c_1 і c_2). Наприклад, розглянемо квадратичну функцію $f(n) = an^2 + bn + c$, де a , b і c – деякі константи, й $a > 0$. Відкидаючи члени молодших порядків і коефіцієнт при старшому члені, знаходимо, що $f(n) = \Theta(n^2)$.

Щоб переконатися в цьому формально, можна прийняти, що $c_1 = \frac{a}{4}$,

$$c_2 = \frac{7a}{4} \text{ і } n_0 = 2 \max \left(\frac{|b|}{a}, \sqrt{\frac{|c|}{a}} \right).$$

Взагалі, для будь-якого полінома $p(n)$ степеня d з додатним старшим

коефіцієнтом маємо $p(n) = \Theta(n^d)$.

Згадаємо важливий окремий випадок використання Θ -позначень:

$\Theta(1)$ позначає обмежену функцію, відділену від нуля якоюсь додатною константою при достатньо великих значеннях аргументу.

1.2.2. Асимптотичні оцінки Θ , O та Ω

Запис $f(n) = \Theta(g(n))$ містить дві оцінки: верхню й нижню. Їх можна розділити.

Якщо $f(n)$ й $g(n)$ – деякі функції, то

- запис $f(n) = O(g(n))$ означає, що знайдеться така константа $c > 0$ й таке n_0 , що $f(n) \leq cg(n)$ для всіх $n \geq n_0$;

- запис $f(n) = \Omega(g(n))$ означає, що знайдеться така константа $c > 0$ й таке n_0 , що $0 \leq cg(n) \leq f(n)$ для всіх $n \geq n_0$.

Теорема 1.1.

1. Для будь-яких двох функцій $f(n)$ і $g(n)$ властивість $f(n) = \Theta(g(n))$ справджується тоді і тільки тоді, коли $f(n) = O(g(n))$ й $f(n) = \Omega(g(n))$
2. Для будь-яких двох функцій $f(n)$ і $g(n)$ властивість $f(n) = O(g(n))$ справджується тоді й тільки тоді, коли $g(n) = \Omega(f(n))$.

Оскільки $an^2 + bn + c = \Theta(n^2)$ (при $a > 0$). Тому $an^2 + bn + c = O(n^2)$.

Працюючи із символами O , Θ і Ω ми маємо справу з *однобічними* рівностями – ці символи можуть стояти тільки праворуч від знака $=$.

Властивості асимптотичних функцій

Введені нами визначення мають деякі властивості транзитивності, рефлексивності й симетричності.

Транзитивність

$f(n) = \Theta(g(n))$ і $g(n) = \Theta(h(n))$ спричиняє $f(n) = \Theta(h(n))$

$f(n) = O(g(n))$ і $g(n) = O(h(n))$ спричиняє $f(n) = O(h(n))$

$f(n) = \Omega(g(n))$ і $g(n) = \Omega(h(n))$ спричиняє $f(n) = \Omega(h(n))$

Рефлексивність

$f(n) = \Theta(f(n))$, $f(n) = O(f(n))$, $f(n) = \Omega(f(n))$

Симетричність

$f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$

Контрольні запитання

1. Які класи росту функцій вам відомі?
2. Доведіть справедливості нерівності для верхньої оцінки складності: $2n^2 \neq O(n)$.
3. Доведіть справедливості рівності для оцінки складності: $2n^2 + n = O(n^2)$.
4. Наведіть приклад алгоритму з алгоритмічною складністю $O(n^2)$.
5. Яку функцію задає нерівність $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$?

Класи задач P і NP

Алгоритми поліноміальної складності (клас P)

Більшість алгоритмів мають поліноміальний порядок складності.

Складність $O(n)$. Іноді час роботи виявляється лінійним $O(n)$, як при послідовному пошуку: при подовженні списку даних удвічі алгоритм працює удвічі довше. В алгоритмах послідовного пошуку нас цікавить процес перегляду списку в пошуках деякого елемента, називаного цільовим. При послідовному пошуку передбачається, що список не відсортований. Наприклад, ключове значення може бути номером співробітника, прізвищем або будь-яким іншим унікальним ідентифікатором. Алгоритм послідовного пошуку послідовно переглядає по одному елементу списку, починаючи з першого, доти, поки не знайде потрібний елемент. Очевидно, що чим далі в списку перебуває конкретне значення ключа, тим більше часу піде на його пошук.

Складність $O(n^2)$. Дуже часто трапляються алгоритми складності $O(n^2)$ – таку складність мають деякі алгоритми сортування: якщо довжину вхідного списку подвоїти, то час роботи алгоритму зросте у 4 рази. Усі вісім існуючих алгоритмів сортування демонструють широкий спектр можливих варіантів поведінки. *Сортування вставками* сортує список, вставляючи черговий елемент у потрібне місце вже відсортованого списку. *Бульбашкове сортування* порівнює елементи попарно, переставляючи між собою елементи тих пар, порядок у яких порушений. *Сортування Шелла* є багатопрохідним сортуванням, при якому список розбивається на підсписки, кожний з яких сортується окремо, причому на кожному проході кількість підсписків зменшується, а їх довжина зростає.

Розглянемо найбільш типовий варіант сортування – бульбашкове сортування. Алгоритм бульбашкового сортування здійснює кілька проходів за списком. При кожному проході відбувається порівняння сусідніх елементів. Якщо порядок сусідніх елементів неправильний, вони міняються місцями. Кожний прохід починається з початку списку. Спочатку порівнюються 1 і 2 елементи, потім 2 і 3, потім 3 і 4 і т. д. Елементи з неправильним порядком у парі переставляються. При виявленні на першому проході найбільшого елемента списку він буде переставлятися з усіма наступними, поки не дійде до кінця списку. Тому при другому проході немає необхідності порівняння з останнім елементом. При другому проході другий за величиною елемент списку опуститься на другу позицію з кінця і т. д. Зазначимо, що при кожному проході ближче до свого місця просувається відразу кілька елементів, хоча гарантовано займає остаточне положення лише один.

Скільки порівнянь виконується в найгіршому випадку? На першому проході буде виконано $n - 1$ порівнянь сусідніх значень, на другому $n - 2$ порівняння. Подальше дослідження показує, що при кожному черговому проході число порівнянь зменшується на 1. Тому складність у найгіршому випадку задають формулою

$$W(n) = \sum_{i=n-1}^1 i = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} = \frac{n^2 - n}{2} \approx \frac{n^2}{2} = O(n^2)$$

Складність стандартного алгоритму матричного множення дорівнює $O(n^3)$ й при збільшенні розмірів матриць удвічі такий алгоритм працює у 8 разів довше.

Матриця – математичний об’єкт, еквівалентний двовимірному масиву. Якщо число стовпців у першій матриці збігається із числом рядків у другій, то ці дві матриці можна перемножити. Для обчислення добутку двох матриць кожний рядок першої почленно множиться на кожний стовпець другої. Потім обчислюється сума таких добутків і записується у відповідну клітинку результату.

Стандартний алгоритм множення матриці розміром $a \times b$ на матрицю розміром $b \times c$ виконує abc множень і $a(b-1)c$ додавань.

Однак дослідникам удалося виявити інші алгоритми, що множать матриці більш ефективно, зокрема, алгоритм Виноградова й алгоритм Штрассена.

Алгоритм Штрассена працює з квадратними матрицями. Насправді він настільки ефективний, що іноді доцільно розширити матриці до квадратних, і при цьому він все одно дає кращий результат. Аналіз загального випадку показує, що число множень при множенні двох $n \times n$ матриць приблизно дорівнює $n^{2.81}$, а число додавань $6n^{2.81} - 6n^2$.

Зводячи три результати до купи, маємо наступну таблицю:

Таблиця 2

Кількість операцій у алгоритмах множення матриць

	Множення	Додавання
Стандартний алгоритм	n^3	$n^3 - n^2$

Алгоритм Виноградова	$\frac{n^3 + 2n^2}{2}$	$\frac{3n^3 + 4n^2 - 4n}{2}$
Алгоритм Штрассена	$n^{2.81}$	$6n^{2.81} - 6n^2$

Усі розглянуті алгоритми мають поліноміальну складність. Найбільш часомістким є алгоритм множення матриць, його складність $O(n^3)$. Головне, однак, те, що ми могли знайти такий розв’язок задач за прийнятний проміжок часу. Усі ці задачі належать до класу P – класу задач поліноміальної складності. Такі задачі називають також практично розв’язними.

Алгоритми недетермінованої поліноміальної складності (клас NP задач)

Крім практично розв’язних задач, що належать до класу P – класу задач поліноміальної складності, існує й інший клас задач: вони практично нерозв’язні й ми не знаємо алгоритмів, здатних розв’язати їх за прийнятний час. Ці задачі утворюють клас NP – недетермінованої поліноміальної складності.

Відзначимо тільки, що складність усіх відомих детермінованих алгоритмів, що вирішують ці задачі, або експоненціальна, або факторіальна. Складність деяких з них дорівнює 2^n , де n – кількість вхідних даних. У цьому випадку при додаванні до списку вхідних даних одного елемента час роботи алгоритму подвоюється. Якщо для розв’язування такої задачі на вході з 10 елементів алгоритму було потрібно 1024 операцій, то на вході з 11 елементів число операцій становитиме 2048. Це значне зростання часу при невеликому збільшенні кількості вхідних даних.

Термін «недетерміновані поліноміальні», що характеризує задачі із класу NP , пояснюється наступним двокроковим підходом до їх розв’язування. На першому кроці є недетермінований алгоритм, що генерує можливий розв’язок такої задачі – щось подібне до спроби вказати розв’язок; іноді така

спроба виявляється успішною, і ми одержуємо оптимальну або близьку до оптимальної відповідь, але частіше відповідь далека від оптимальної. На другому кроці перевіряється, чи дійсно відповідь, отримана на першому кроці, є розв'язком початкової задачі. Кожний із цих кроків окремо вимагає поліноміального часу. Проблема, однак, у тому, що ми не знаємо, скільки разів нам доведеться повторити обидва кроки, щоб одержати шуканий розв'язок. Хоча обидва кроки є поліноміальними, кількість звертань до них може бути експоненціальною або факторіальною.

Задача про комівояжера

До класу *NP* належить *задача про комівояжера*. Нам заданий набір міст і «вартість» подорожі між будь-якими двома з них. Потрібно визначити такий порядок, при якому слід відвідати всі міста (по одному разу) і повернутися у початкове місто, щоб загальна вартість подорожі виявилася мінімальною. Цю задачу можна застосувати, наприклад, для визначення порядку ефективного збору сміття з баків на вулицях міста або вибору найкоротшого шляху поширення інформації із усіх вузлів комп'ютерної мережі. Вісім міст можна впорядкувати 40 320 можливими способами, а для десяти міст це число зростає вже до 3 628 800. Пошук найкоротшого шляху вимагає перебору всіх цих можливостей. Припустимо, що в нас є алгоритм, здатний підрахувати вартість подорожі через 15 міст у зазначеному порядку. Якщо за секунду такий алгоритм здатний пропустити через себе 100 варіантів, то йому буде потрібно більше чотирьох століть, щоб дослідити всі можливості й знайти найкоротший шлях. Навіть якщо в нас є 400 комп'ютерів, для обчислень знадобиться рік, хоча ми маємо лише 15 міст. Для 20 міст мільярд комп'ютерів працюватимуть паралельно протягом дев'яти місяців, щоб знайти найкоротший шлях. Ясно, що швидше й дешевше подорожувати хоч як-небудь, ніж чекати, поки комп'ютери дадуть оптимальний розв'язок.

Чи можна знайти найкоротший шлях, не переглядаючи їх всі? Дотепер нікому не вдалося створити алгоритм, який не займається, по суті, переглядом

усіх шляхів. Коли число міст невелике, задача вирішується швидко, однак це не означає, що так буде завжди, а нас цікавить саме розв'язок загальної задачі.

Задача про комівояжера, звичайно, дуже схожа на задачі про графи. Кожне місто можна представити вершиною графа, наявність шляху між двома містами – ребром, вартість подорожі між ними – вагою цього ребра. Звідси можна зробити висновок, що алгоритм пошуку найкоротшого шляху вирішує й задачу комівояжера, однак це не так. Які ж дві умови задачі про комівояжера відрізняють її від задачі про найкоротший шлях? По-перше, ми повинні відвідати всі міста, а алгоритм пошуку найкоротшого шляху дає лише шлях між двома заданими містами. Якщо вибрати шлях з найкоротших відрізків, видаваних алгоритмом пошуку найкоротших шляхів, то він буде проходити через деякі міста по декілька разів. Друга відмінність полягає у вимозі повернення у початкову точку, така вимога відсутня у пошуку найкоротшого шляху.

З огляду на те, наскільки велике число можливих упорядкувань вершин, детермінований алгоритм, що порівнює всі можливі способи упорядкування, працює надто довго. Щоб показати, що ця задача належить до класу NP , необхідно зрозуміти, як її можна розв'язати за допомогою описаної вище двокрокової процедури. У задачі про комівояжера на першому кроці випадковим чином генерується деяке упорядкування міст. Оскільки це недетермінований процес, щоразу виходитиме новий порядок. Очевидно, що процес генерації можна реалізувати за поліноміальний час: ми можемо зберігати список міст, генерувати випадковий номер, вибирати зі списку місто з цією назвою і видаляти його зі списку, щоб він не з'явився другий раз. Така процедура виконується за $O(n)$ операцій, де n – число міст. На другому кроці відбувається підрахунок вартості подорожі по містах у зазначеному порядку. Для цього потрібно просто додати вартості подорожі між послідовними парами міст у списку, що також вимагає $O(n)$ операцій. Обидва кроки

поліноміальні, тому задача про комівояжера належить до класу NP . Часомісткою робить її саме необхідне число ітерацій цієї процедури.

Слід зазначити, що таку двокрокову процедуру можна було застосувати до кожної із задач, що розглядалися нами раніше. Наприклад, сортування списку можна виконувати, генеруючи довільний порядок елементів початкового списку й перевіряючи, чи не є цей порядок зростаючим.

Чи не відносить це міркування задачу сортування до класу NP ? Звичайно, відносить. Різниця між класом P и класом NP у тому, що в першому випадку наявним є детермінований алгоритм, що вирішує задачу за поліноміальний час, а в другому ми такого алгоритму не знаємо.

Зведення задачі до іншої задачі

Один зі способів розв'язування задач полягає в тому, щоб звести, або редукувати, одну задачу до іншої. Тоді алгоритм розв'язування другої задачі можна перетворити таким чином, щоб він вирішував першу. Якщо перетворення виконується за поліноміальний час і друга задача вирішується за поліноміальний час, то й наша нова задача також вирішується за поліноміальний час.

Пояснимо наше міркування прикладом.

Нехай перше завдання полягає в тому, щоб повернути значення «так» у випадку, якщо одна з даних булевих змінних має значення «істина», і повернути «ні» у протилежному випадку.

Друга задача полягає в тому, щоб знайти максимальне значення в списку цілих чисел. Кожна з них допускає простий зрозумілий розв'язок, але припустимо на хвилину, що ми знаємо розв'язок задачі про пошук максимуму, а задачу про булеві змінні вирішувати не вміємо.

Ми прагнемо звести задачу про булеві змінні до задачі про максимум цілих чисел. Напишемо алгоритм перетворення набору значень булевих змінних у список цілих чисел, який значенню «false» ставить у відповідність число 0, а значенню «true» – число 1.

Потім скористаємося алгоритмом пошуку максимального елемента в списку. По тому, як складався список, робимо висновок, що цей максимальний елемент може бути або нулем, або одиницею. Таку відповідь можна перетворити у відповідь у задачі про булеві змінні, повертаючи «так», якщо максимальне значення дорівнює 1, і «ні», якщо воно дорівнює 0.

Пошук максимального значення виконується за лінійний час, а редукція першої задачі до другої теж вимагає лінійного часу, тому задачу про булеві змінні теж можна розв'язати за лінійний час.

Типові *NP* задачі

Кожна із задач у 1.3.5 є або оптимізаційною, або задачею про прийняття рішення. Метою оптимізаційної задачі зазвичай є конкретний результат, що представляє собою мінімальне або максимальне значення. У задачі про прийняття рішення зазвичай задають деяке граничне значення, і нас цікавить, чи існує розв'язок, більший (у задачах максимізації) або менший (у задачах мінімізації) зазначеної границі. Відповіддю в задачах оптимізації є отриманий конкретний результат, а в задачах про прийняття рішень – «так» або «ні».

У 1.3.3 розглянутий оптимізаційний варіант задачі про комівояжера. Це задача мінімізації, і нас цікавив шлях мінімальної вартості. У варіанті ухвалення рішення ми могли б запитати, чи існує шлях комівояжера з вартістю, яка є меншою за задану константу C . Ясно, що відповідь у задачі про ухвалення рішення залежить від обраної границі. Якщо ця границя дуже велика (наприклад, вона перевищує сумарну вартість усіх доріг), то відповідь «так» одержати нескладно. Якщо ця границя надто мала (наприклад, вона менша за вартість шляху між будь-якими двома містами), то відповідь «ні» також дається легко. У інших проміжних випадках час пошуку відповіді дуже великий й порівнянний з часом розв'язування оптимізаційної задачі. Тому ми розглядатимемо як задачі оптимізації, так і задачі прийняття рішень, використовуючи ту з них, яка точніше відповідає нашим поточним цілям.

Наведемо опис шести NP задач – як в оптимізаційному варіанті, так і у варіанті прийняття рішення.

Розфарбування графа

Граф $G = (V, E)$ є набором вершин, або вузлів, V і набором ребер E , що з'єднують вершини попарно. Будемо розглядати тільки *неорієнтовані* графи. Вершини графа можна розфарбувати в різні кольори, які зазвичай позначають цілими числами. Нас цікавлять такі розфарбування, у яких кінці кожного ребра пофарбовані різними кольорами. Очевидно, що в графі з N вершинами можна пофарбувати вершини в N різних кольорів, але чи можна обійтися меншою кількістю кольорів?

У задачі оптимізації нас цікавить мінімальне число кольорів, необхідних для розфарбування вершин графа [9].

У задачі ухвалення рішення нас цікавить, чи можна розфарбувати вершини в 3 або менше кольорів.

У задачі про розфарбування графа є практичні додатки. Якщо кожна вершина графа позначає курс, який читають у коледжі, і вершини з'єднуються ребром, якщо ці курси читають для одного і того ж студента, то отриманий граф досить складний. Якщо припустити, що кожному студенту читають 5 курсів, то на нього припадає 10 ребер. Припустимо, що на 3500 студентів припадає 500 курсів. Тоді в графі буде 500 вершин і 35000 ребер. Якщо на іспити відведено 20 днів, то це означає, що вершини графа потрібно розфарбувати в 20 кольорів, щоб у жодного студента не було по два іспити на день.

Розробка безконфліктного розкладу іспитів еквівалентна розфарбуванню графів. Однак задача розфарбування графів належить до класу NP , тому розробка безконфліктного розкладу за прийнятний час неможлива. Крім того, при плануванні іспитів зазвичай потрібно, щоб у студента було не більше двох іспитів на день, а екзамени з різних частин курсу призначаються в один день. Очевидно, що розробка «досконалого» плану іспитів неможлива, і тому необхідна інша техніка для одержання, принаймні, непоганих розкладів.

Розкладка по ящиках

Нехай у нас є кілька ящиків одиничної ємності й набір об'єктів різних розмірів $s_1, \dots, s_N$. У задачі оптимізації нас цікавить найменша кількість ящиків, необхідна для розкладки всіх об'єктів, а в задачі ухвалення рішення – чи можна упакувати всі об'єкти у B або меншу кількість ящиків.

Ця задача виникає при запису інформації на диск або у фрагментовану пам'ять комп'ютера, при ефективному розподілі вантажу на кораблях, при вирізанні частин зі стандартних порцій матеріалу на замовлення клієнтів. Якщо, наприклад, у нас є великі металеві пластини й список замовлень на менші пластини, то, природно, ми прагнемо розподілити замовлення якомога щільніше, зменшивши втрати й збільшивши прибуток.

Упакування рюкзака

У нас є набір об'єктів об'ємом $s_1, \dots, s_N$ вартості $w_1, \dots, w_N$. У задачі оптимізації ми прагнемо впакувати рюкзак об'ємом K так, щоб його вартість була максимальною. У задачі ухвалення рішення нас цікавить, чи можна досягти того, щоб сумарна вартість упакованих об'єктів була, щонайменше, W .

Ця задача виникає при виборі стратегії вкладення грошей: об'ємом тут є обсяг різних вкладень, вартістю – передбачувана величина доходу, а об'єм рюкзака визначається розміром (обсягом) запланованих капіталовкладень.

Задача про суми елементів підмножин

Нехай у нас є множина об'єктів різних розмірів $s_1, \dots, s_N$ і деяка додатна верхня границя L . У задачі оптимізації нам необхідно знайти набір об'єктів, сума розмірів яких найбільш близька до L і не перевищує цієї верхньої границі. У задачі ухвалення рішення потрібно встановити, чи існує набір об'єктів із сумою розмірів L . Це спрощена версія задачі про упакування рюкзака.

Задача про істинність КНФ-виразу

Кон'юнктивна нормальна форма (КНФ) є послідовністю булевих виразів, пов'язаних між собою операторами AND (позначуваними $\wedge$),

причому кожен вираз є мономом від булевих змінних або їх заперечень, пов'язаних операторами OR (які позначають через $\vee$).

Приклад 1.9. Булевий вираз у кон'юнктивній нормальній формі (заперечення позначають рискою над іменем змінної):

$$(a \vee b) \wedge (\bar{a} \vee c) \wedge (a \vee b \vee \bar{c} \vee d) \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (a \vee \bar{b} \vee \bar{c} \vee \bar{d} \vee e).$$

Задача про істинність булевого виразу в кон'юнктивній нормальній формі ставиться тільки у варіанті прийняття рішення: чи існують у змінних, що входять у вираз, такі значення істинності, підстановка яких робить увесь вираз дійсним. Як число змінних, так і складність виразу не обмежені, тому число комбінацій значень істинності може бути дуже великим.

Задача планування робіт

Нехай у нас є набір робіт і ми знаємо час, необхідний для завершення кожної з них, $t_1, t_2, \dots, t_N$, терміни $d_1, d_2, \dots, d_N$, до яких ці роботи повинні бути обов'язково завершені, а також штрафи $p_1, p_2, \dots, p_N$, які будуть накладені при незавершенні кожної роботи у встановлений термін. Задача оптимізації вимагає встановити порядок робіт, який мінімізує штрафи, що накладаються. У задачі прийняття рішень ми запитуємо, чи існує порядок робіт, при якому величина штрафу буде не більше, ніж P .

Контрольні запитання

1. Яку обчислювальну складність має алгоритм сортування Шелла?
2. Яка обчислювальна складність алгоритму множення матриць?
3. Чому алгоритм розв'язування задачі про комівояжера має недетерміновану поліноміальну обчислювальну складність?
4. Який типовий підхід до розв'язування задач NP -класу вам відомий?
5. Назвіть відомі вам задачі, розв'язування яких приводить до алгоритмів з недетермінованою поліноміальною обчислювальною складністю.